Zero-Knowledge Identity: Privacy-Preserving Authentication

Pars Network Technical Whitepaper PNP-006

Cyrus Pahlavi, Pars Team research@pars.network Version 1.0 — February 2026

Abstract

We present the Pars Zero-Knowledge Identity (ZK-ID) system, a privacy-preserving authentication framework based on zk-SNARKs and anonymous credentials. ZK-ID enables users to prove possession of identity attributes—such as age range, citizenship status, or governance eligibility—without revealing any additional information. The system uses Groth16 proofs [1] for efficient verification on the Pars L2 rollup, with a credential issuance protocol based on BBS+ signatures [2] that supports selective disclosure and unlinkable multi-show presentations. We formalise the system's privacy and soundness properties, prove that credential presentations are zero-knowledge and unlinkable under the q-SDH assumption, and demonstrate that proof generation takes under 2 s on mobile hardware with verification under 10 ms on-chain. ZK-ID integrates with the Pars DID system (PNP-008) for credential anchoring and with the session protocol (PNP-003) for anonymous session establishment.

Keywords: zero-knowledge proofs, anonymous credentials, selective disclosure, zk-SNARKs, BBS+ signatures, privacy-preserving authentication

Contents

1 Introduction **4**
1.1 Requirements . . . 4
1.2 Contributions . . . 4

2 Preliminaries **4**
2.1 Bilinear Pairings . . . 4
2.2 BBS+ Signatures . . . 4
2.3 zk-SNARKs . . . 5

3 Credential Scheme **5**
3.1 Credential Structure . . . 5
3.2 Credential Issuance . . . 6
3.3 Selective Disclosure . . . 6

4 Predicate Proofs **7**
4.1 Range Proofs . . . 7
4.2 Set Membership Proofs . . . 7
4.3 Compound Predicates . . . 7

5 On-Chain Verification **8**
5.1 Groth16 Circuit . . . 8
5.2 Gas Cost Analysis . . . 9

6 Privacy-Preserving Revocation **9**
6.1 Accumulator-Based Revocation . . . 9
6.2 Privacy Properties . . . 10

7 Integration **10**
7.1 DID Integration . . . 10
7.2 Governance Integration . . . 11
7.3 Session Establishment . . . 11

8 Performance Analysis **11**

9 Security Analysis **11**
9.1 Assumptions 11
9.2 Security Properties 12

10 Issuer Architecture **12**
10.1 Issuer Types 12
10.2 Issuer Key Management 13

11 Related Work **13**

12 Conclusion **13**

A Circuit Specification **14**

B Credential Encoding **15**

1 Introduction

Identity is the gateway to every digital service: governance participation, financial transactions, communication access. Traditional identity systems operate on an all-or-nothing disclosure model: a user must reveal their full identity to prove a single attribute. For diaspora members, this creates an unacceptable tradeoff between access and privacy.

A user who wishes to vote in Pars governance must prove they are an eligible community member, but need not reveal their name, address, or specific citizenship. A user who wishes to access age-restricted services need only prove age ≥ 18, not their exact birthdate.

1.1 Requirements

R1: Selective Disclosure: Prove individual attributes without revealing others.

R2: Unlinkability: Multiple presentations of the same credential are unlinkable.

R3: Non-Transferability: Credentials cannot be shared or delegated.

R4: Revocability: Issuers can revoke credentials without breaking privacy.

R5: On-Chain Verification: Proofs verifiable by L2 smart contracts.

R6: Offline Issuance: Credentials obtainable without real-time issuer interaction.

1.2 Contributions

- Complete ZK-ID specification: issuance, presentation, and verification.
- BBS+ credential scheme with selective disclosure and unlinkability.
- Groth16 circuit for on-chain credential verification.
- Privacy-preserving revocation via cryptographic accumulators.
- Performance analysis on mobile and L2 environments.

2 Preliminaries

2.1 Bilinear Pairings

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be cyclic groups of prime order p with generators g_1, g_2. A bilinear pairing is $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ satisfying:

1. **Bilinearity:** $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$
2. **Non-degeneracy:** $e(g_1, g_2) \neq 1$
3. **Computability:** e is efficiently computable

2.2 BBS+ Signatures

Definition 2.1 (BBS+ Signature [2]). *For messages $(m_1, \ldots, m_L)$, signer key pair $(x, w = g_2^x)$, and public parameters $(h_0, h_1, \ldots, h_L) \in \mathbb{G}_1$:*

Sign: choose random $e, s \leftarrow \mathbb{Z}_p$; compute

$$A = \Big(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i}\Big)^{1/(x+e)}$$

Signature: $\sigma = (A, e, s)$.
Verify: check $e(A, w \cdot g_2^e) = e\big(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i},\ g_2\big)$.

2.3 zk-SNARKs

Definition 2.2 (Groth16 [1]). *A zk-SNARK for arithmetic circuit C with public input x and witness w consists of:*

- $\mathrm{Setup}(C) \to (\mathrm{pk}, \mathrm{vk})$*: circuit-specific setup*
- $\mathrm{Prove}(\mathrm{pk}, x, w) \to \pi$*: proof generation*
- $\mathrm{Verify}(\mathrm{vk}, x, \pi) \to \{0, 1\}$*: verification*

Properties: completeness, knowledge soundness, zero-knowledge. Proof size: 3 group elements (≈ 128 bytes on BN254). Verification: 1 pairing + 1 multi-exponentiation.

3 Credential Scheme

3.1 Credential Structure

Definition 3.1 (Pars Credential). *A credential $\mathcal{C}$ consists of:*

- *Attribute vector* $\mathbf{m} = (m_1, \ldots, m_L)$
- *BBS+ signature* $\sigma = (A, e, s)$ *from issuer*
- *Holder secret* sk_H *(bound to attribute* m_1*)*
- *Revocation handle* rh

Standard attribute schema:

Table 1: Credential Attribute Schema

Index	Attribute	Type
m_1	Holder secret key	$\mathbb{Z}_p$
m_2	Issuer identifier	$\mathbb{Z}_p$
m_3	Issuance date	uint64
m_4	Expiry date	uint64
m_5	Credential type	enum
m_6	Age (years)	uint8
m_7	Citizenship hash	$\mathbb{Z}_p$
m_8	Governance weight	uint32
m_9	Revocation handle	$\mathbb{Z}_p$
m_{10}	Extension field	bytes

3.2 Credential Issuance

Algorithm 1 Credential Issuance Protocol

Require: Holder H with secret sk_H, Issuer I with key (x, w)

▷ Phase 1: Holder commitment

H computes $C_H = h_1^{\mathrm{sk}_H} \cdot h_0^{s_1}$ (Pedersen commitment)

H sends C_H and ZK proof of knowledge of (sk_H, s_1) to I

▷ Phase 2: Issuer signing

I verifies ZK proof

I sets attributes $m_2, \ldots, m_L$ based on identity verification

I chooses $e, s_2 \leftarrow \mathbb{Z}_p$

I computes: $B = C_H \cdot h_0^{s_2} \cdot \prod_{i=2}^{L} h_i^{m_i}$

I computes: $A = (g_1 \cdot B)^{1/(x+e)}$

I sends $(A, e, s_2, m_2, \ldots, m_L)$ to H

▷ Phase 3: Holder finalisation

H sets $s = s_1 + s_2$

H verifies: $e(A, w \cdot g_2^e) = e\big(g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i},\ g_2\big)$

H stores $\mathcal{C} = (\mathbf{m}, (A, e, s))$

The issuer never learns sk_H, ensuring the credential is bound to the holder.

3.3 Selective Disclosure

Definition 3.2 (Selective Disclosure Proof)**.** *Given credential $\mathcal{C}$ with attributes $\mathbf{m}$, disclosed set $D \subseteq [L]$, and hidden set $H = [L] \setminus D$:*

The holder proves knowledge of $(A, e, s, \{m_i\}_{i \in H})$ such that $\sigma = (A, e, s)$ is a valid BBS+ signature on $(\{m_i\}_{i \in D} \cup \{m_i\}_{i \in H})$, without revealing $(A, e, s, \{m_i\}_{i \in H})$.

Algorithm 2 Selective Disclosure Presentation

Require: Credential $\mathcal{C}$, disclosed set D, verifier nonce n

▷ Randomise signature for unlinkability

$r_1, r_2 \leftarrow \mathbb{Z}_p$

$A' = A \cdot g_1^{r_1}$

$\bar{A} = (A')^{-e} \cdot (g_1 \cdot h_0^s \cdot \prod_{i=1}^{L} h_i^{m_i})^{r_1}$

$d = h_0^{s - r_1 \cdot r_2} \cdot \prod_{i \in H} h_i^{m_i}$

▷ Generate Schnorr proof for hidden values

Choose random $\tilde{e}, \tilde{s}, \{\tilde{m}_i\}_{i \in H}$

Compute commitments T_1, T_2, T_3 from randomness

$c = H(A', \bar{A}, d, T_1, T_2, T_3, \{m_i\}_{i \in D}, n)$

Compute responses: $\hat{e} = \tilde{e} + c \cdot e$, etc.

return proof $\pi = (A', \bar{A}, d, c, \hat{e}, \hat{s}, \{\hat{m}_i\}_{i \in H})$

Theorem 3.1 (Zero-Knowledge)**.** *The selective disclosure proof reveals no information about hidden attributes or the original signature beyond what is deducible from the disclosed attributes.*

Proof. The randomisation $(A', \bar{A})$ ensures the presented values are uniformly distributed and unlinkable to (A, e, s). The Schnorr proof for hidden values is zero-knowledge by construction

(the simulator can produce identically distributed proofs without knowing the witness). The only information revealed is the set of disclosed attributes $\{m_i\}_{i \in D}$ and the fact that a valid BBS+ signature exists. □

Theorem 3.2 (Unlinkability)**.** *Two presentations of the same credential are computationally unlinkable.*

Proof. Each presentation uses fresh randomness (r_1, r_2), producing new $(A', \bar{A}, d)$. Under DDH in $\mathbb{G}_1$, the randomised values are indistinguishable from random group elements. The Schnorr proof components are similarly fresh. No deterministic function of the credential appears in either presentation. □

4 Predicate Proofs

Beyond selective disclosure, ZK-ID supports predicate proofs on hidden attributes.

4.1 Range Proofs

Definition 4.1 (Age Range Proof)**.** *Prove $m_6 \geq 18$ without revealing m_6, using a Bulletproofs-style range proof [3] integrated into the BBS+ presentation.*

Algorithm 3 Range Predicate Proof

Require: Hidden attribute m_i, predicate $m_i \geq v$

1: $\delta = m_i - v$ ▷ $\delta \geq 0$ iff predicate holds
2: Decompose δ into bits: $\delta = \sum_{j=0}^{n-1} b_j 2^j$
3: For each bit b_j: commit $C_j = g^{b_j} h^{r_j}$
4: Prove each $b_j \in \{0, 1\}$ via Schnorr OR proof
5: Prove $\sum_j b_j 2^j = m_i - v$ via inner-product argument
6: Combine with BBS+ presentation proof

4.2 Set Membership Proofs

Definition 4.2 (Citizenship Proof)**.** *Prove $m_7 \in S$ where S is a set of approved citizenship hashes, without revealing which element of S matches.*

This uses an accumulator-based membership proof [4]:

1. Issuer publishes a cryptographic accumulator acc over the valid citizenship hash set S.
2. The holder provides a witness w proving $m_7 \in \text{acc}$.
3. Verification checks $e(m_7, w) = e(\text{acc}, g_2)$ (for pairing-based accumulators).

4.3 Compound Predicates

Multiple predicates can be combined in a single proof:

Listing 1: Compound Predicate Specification

```go
type ProofRequest struct {
    // Attributes to disclose
    Disclosed []int
    // Range predicates
    Ranges []RangePredicate
```

```
    // Set membership predicates
    Memberships []MembershipPredicate
    // Equality predicates across credentials
    Equalities []EqualityPredicate
    // Verifier nonce
    Nonce [32]byte
}

type RangePredicate struct {
    AttrIndex int
    Operator  string // ">=", "<=", "=="
    Value     int64
}

type MembershipPredicate struct {
    AttrIndex   int
    Accumulator []byte
}

type EqualityPredicate struct {
    Cred1Attr int
    Cred2Attr int
}
```

5 On-Chain Verification

5.1 Groth16 Circuit

For on-chain verification (e.g., governance eligibility), we compile the BBS+ verification into a Groth16 circuit:

Definition 5.1 (ZK-ID Verification Circuit)**.** *Public inputs:* $(D_{\text{attrs}}, \text{issuerPK}, \text{predicateHash}, \text{nonce})$.
Private witness: $(\sigma, \{m_i\}_{i \in H}, \text{accWitness})$.
Circuit constraints:

1. *BBS+ signature verification*
2. *Predicate satisfaction (range, membership)*
3. *Revocation non-membership (accumulator check)*
4. *Nonce binding (prevents replay)*

Listing 2: On-Chain ZK-ID Verifier

```
contract ZKIDVerifier {
    // Groth16 verification key (circuit-specific)
    struct VerifyingKey {
        uint256[2] alpha;
        uint256[2][2] beta;
        uint256[2][2] gamma;
        uint256[2][2] delta;
        uint256[2][] ic;
    }

    VerifyingKey public vk;

```

```
    function verifyCredential(
        uint256[8] calldata proof,
        uint256[] calldata publicInputs
    ) external view returns (bool) {
        // Decode Groth16 proof
        // (A, B, C) from proof array
        // Verify pairing equation:
        // e(A, B) = e(alpha, beta) *
        //   e(sum(ic[i]*input[i]), gamma) *
        //   e(C, delta)
        return _verifyProof(
            proof, publicInputs
        );
    }

    function verifyAndExecute(
        uint256[8] calldata proof,
        uint256[] calldata publicInputs,
        bytes calldata action
    ) external {
        require(
            verifyCredential(proof, publicInputs),
            "Invalid credential proof"
        );
        // Execute gated action
        _executeAction(action, publicInputs);
    }
}
```

5.2 Gas Cost Analysis

Table 2: On-Chain Verification Gas Costs

Operation	**Gas**
Groth16 proof verification	230 000
Public input decoding	5 000
Nonce uniqueness check	20 000
Action execution (typical)	50 000
Total	**305 000**

6 Privacy-Preserving Revocation

6.1 Accumulator-Based Revocation

Credential revocation uses a dynamic accumulator [4]:

Definition 6.1 (Revocation Accumulator)**.** *The revocation accumulator V contains revocation handles of* non-revoked *credentials. To prove non-revocation, a holder provides a witness w such that:*

$$e(V, g_2) = e(w, g_2^{\mathrm{rh}} \cdot g_2')$$

where rh *is the holder's revocation handle.*

Algorithm 4 Revocation Check in Proof

Require: Credential with handle rh, current accumulator V, witness w

1: Holder updates witness w with latest accumulator changes
2: Holder includes in ZK proof:
   Prove knowledge of (rh, w) such that
   $e(V, g_2) = e(w, g_2^{\mathrm{rh}} \cdot g_2')$
3: Verifier checks ZK proof and that V matches current epoch

The accumulator V is published on-chain each epoch. Holders update their witnesses locally using published accumulator deltas.

6.2 Privacy Properties

Proposition 6.1 (Revocation Privacy)**.** *The revocation check reveals no information about the holder's identity or which specific credential is being checked.*

Proof. The witness w and handle rh are hidden inside the zero-knowledge proof. The verifier learns only that the holder possesses a valid, non-revoked credential—not which specific credential or handle is involved. □

7 Integration

7.1 DID Integration

ZK-ID credentials are anchored to the Pars DID system (PNP-008):

Listing 3: Credential-DID Binding

```go
type ZKIDCredential struct {
    Schema      CredentialSchema
    Attributes  []Attribute
    Signature   BBS_Signature
    IssuerDID   string
    HolderDID   string // blind-linked
    Revocation  RevocationInfo
}

func (c *ZKIDCredential) Present(
    req ProofRequest,
) (*Presentation, error) {
    // Generate selective disclosure proof
    proof, err := c.selectiveDisclose(
        req.Disclosed,
    )
    if err != nil {
        return nil, fmt.Errorf(
            "disclosure: %w", err,
        )
    }
    // Add predicate proofs
    for _, pred := range req.Ranges {
        rp, err := c.rangeProof(
            pred.AttrIndex,
            pred.Operator,
            pred.Value,
        )
```

```
        if err != nil {
            return nil, fmt.Errorf(
                "range proof: %w", err,
            )
        }
        proof.AddPredicate(rp)
    }
    // Add non-revocation proof
    nrp, err := c.nonRevocationProof()
    if err != nil {
        return nil, fmt.Errorf(
            "revocation proof: %w", err,
        )
    }
    proof.AddRevocation(nrp)
    return proof, nil
}
```

7.2 Governance Integration

For governance voting (PNP-005), ZK-ID provides eligibility proofs:

1. Voter presents ZK proof: "I hold a valid, non-revoked Pars credential with governance weight ≥ 1".
2. The proof reveals neither identity nor exact weight.
3. The nullifier (derived from sk_H and election ID) prevents double-voting while preserving anonymity.

Definition 7.1 (Vote Nullifier)**.**

$$\text{nullifier} = H(\mathrm{sk}_H \| \text{electionID})$$

Published on-chain to prevent double voting. Unlinkable across elections because electionID *differs.*

7.3 Session Establishment

ZK-ID enables anonymous session establishment:

1. Party A presents a ZK proof of credential possession.
2. Party B verifies the proof, confirming A is a valid community member.
3. Session proceeds with PSP (PNP-003) without B learning A's identity.

8 Performance Analysis

9 Security Analysis

9.1 Assumptions

1. **q-SDH:** The q-Strong Diffie-Hellman assumption in $\mathbb{G}_1$ (for BBS+ unforgeability).
2. **DDH:** Decisional Diffie-Hellman in $\mathbb{G}_1$ (for unlinkability).
3. **KEA:** Knowledge of Exponent Assumption (for Groth16 soundness).
4. **Random Oracle:** Hash functions modelled as random oracles for Schnorr proofs.

Table 3: ZK-ID Operation Timing

Operation	**Mobile**	**Desktop**
Credential issuance (holder)	120 ms	45 ms
BBS+ selective disclosure (5/10)	85 ms	32 ms
Range predicate proof	340 ms	130 ms
Set membership proof	210 ms	80 ms
Non-revocation proof	180 ms	70 ms
Full Groth16 proof generation	1.8 s	0.7 s
On-chain Groth16 verification	—	8 ms

Table 4: Proof Sizes

Proof Type	**Size**
BBS+ selective disclosure	512 B
Range predicate (32-bit)	672 B
Set membership	320 B
Non-revocation	384 B
Groth16 (on-chain)	128 B
Full presentation	1.5–2 KB

9.2 Security Properties

Theorem 9.1 (Unforgeability)**.** *No PPT adversary can produce a valid ZK-ID presentation without possessing a credential issued by a trusted issuer, under the q-SDH assumption.*

Proof. A valid presentation requires knowledge of a BBS+ signature (A, e, s) on the attribute vector. By the q-SDH unforgeability of BBS+ [2], no PPT adversary can forge such a signature without the issuer's secret key. The Schnorr proof of knowledge ensures the presenter possesses (not merely observed) the signature. □

Theorem 9.2 (Non-Transferability)**.** *Credential presentations are bound to the holder's secret key* sk_H*; delegation requires revealing* sk_H*.*

Proof. Attribute $m_1 = \mathrm{sk}_H$ is signed by the issuer as part of the credential. The presentation requires proving knowledge of sk_H. Delegating the credential requires sharing sk_H, which also compromises all other credentials and sessions bound to it. Rational holders will not delegate. □

10 Issuer Architecture

10.1 Issuer Types

Table 5: ZK-ID Issuer Types

Type	**Example**	**Trust Model**
Community	Pars Council	Elected, on-chain
Institutional	University	Traditional CA
Peer	Community vouching	Web-of-trust
Self-Issued	Self-attestation	No external trust

10.2 Issuer Key Management

Issuer keys use threshold BBS+:

Definition 10.1 (Threshold Issuer)**.** *Issuer key x is shared as $(x_1, \ldots, x_n)$ among n issuance nodes using Shamir's secret sharing. Credential issuance requires t-of-n nodes to contribute partial signatures, which are combined into a valid BBS+ signature.*

This prevents any single issuer node from unilaterally issuing credentials or being a single point of compromise.

11 Related Work

Idemix [5] introduced CL-signature-based anonymous credentials. Hyperledger AnonCreds [6] implements CL signatures for enterprise identity. BBS+ [2] provides shorter signatures and more efficient proofs. Semaphore [7] uses Groth16 for on-chain anonymous signalling. WorldID [8] uses iris biometrics with ZK proofs. Zupass [9] combines PCDs with identity.

ZK-ID differs by combining BBS+ credentials with Groth16 on-chain verification, integrating with a diaspora-specific governance and communication platform, and providing accumulator-based revocation.

12 Conclusion

ZK-ID provides privacy-preserving authentication for the Pars Network. BBS+ credentials enable selective disclosure with unlinkable multi-show presentations. Groth16 proofs allow on-chain verification of credential predicates. Accumulator-based revocation preserves privacy. The system achieves proof generation under 2 s on mobile with on-chain verification under 10 ms and 128 B proof size.

The key insight is that identity verification and identity disclosure are separable concerns. By proving predicates about attributes without revealing the attributes themselves, ZK-ID enables full governance participation while preserving the anonymity essential for diaspora safety.

References

[1] J. Groth, "On the Size of Pairing-Based Non-interactive Arguments," *EUROCRYPT 2016*, pp. 305–326, 2016.

[2] M. H. Au, W. Susilo, and Y. Mu, "Constant-Size Dynamic k-TAA," *SCN 2006*, pp. 111–125, 2006.

[3] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short Proofs for Confidential Transactions," *IEEE S&P*, pp. 315–334, 2018.

[4] J. Camenisch and A. Lysyanskaya, "Dynamic Accumulators and Application to Efficient Revocation," *CRYPTO 2002*, pp. 61–76, 2002.

[5] J. Camenisch and A. Lysyanskaya, "An Efficient System for Non-transferable Anonymous Credentials," *EUROCRYPT 2001*, pp. 93–118, 2001.

[6] Hyperledger Foundation, "AnonCreds Specification v1.0," 2023.

[7] Semaphore, "A Zero-Knowledge Signaling System," *semaphore.appliedzkp.org*, 2020.

[8] Worldcoin, "World ID Protocol," 2023.

[9] 0xPARC, "Zupass: Programmable Cryptographic Documents," 2023.

[10] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct NIZK for a von Neumann Architecture," *USENIX Security*, 2014.

[11] S. Goldwasser, S. Micali, and C. Rackoff, "Knowledge Complexity," *SIAM J. Comput.*, 18(1), 1989.

[12] D. Boneh, X. Boyen, and H. Shacham, "Short Group Signatures," *CRYPTO 2004*, pp. 41–55, 2004.

[13] T. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," *CRYPTO '91*, pp. 129–140, 1991.

[14] C. P. Schnorr, "Efficient Identification and Signatures for Smart Cards," *CRYPTO '89*, pp. 239–252, 1989.

[15] M. Blum, P. Feldman, and S. Micali, "Non-Interactive Zero-Knowledge and Its Applications," *STOC*, pp. 103–112, 1988.

[16] R. Canetti, "Universally Composable Security," *FOCS*, pp. 136–145, 2001.

[17] D. Chaum, "Security Without Identification," *Commun. ACM*, 28(10):1030–1044, 1985.

[18] A. Fiat and A. Shamir, "How To Prove Yourself," *CRYPTO '86*, pp. 186–194, 1986.

[19] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," *ASIACRYPT 2001*, pp. 514–532, 2001.

[20] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, 22(6), 1976.

[21] D. J. Bernstein, "Curve25519," *PKC 2006*, 2006.

[22] G. Wood, "Ethereum Yellow Paper," 2014.

[23] E. Ben-Sasson *et al.*, "Zerocash: Decentralized Anonymous Payments," *IEEE S&P*, pp. 459–474, 2014.

[24] A. Gabizon, Z. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR ePrint 2019/953*, 2019.

[25] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, and H. Wu, "Zexe: Enabling Decentralized Private Computation," *IEEE S&P*, 2020.

[26] W3C, "Verifiable Credentials Data Model v2.0," 2022.

A Circuit Specification

The Groth16 circuit for ZK-ID verification has approximately 50 000 R1CS constraints, broken down as:

Sub-Circuit	**Constraints**
BBS+ signature check	25 000
Range predicate (32-bit)	8 000
Set membership	5 000
Non-revocation	7 000
Nonce binding	500
Nullifier derivation	4 500
Total	**50 000**

B Credential Encoding

Listing 4: Credential Binary Format

```
// 4 bytes: version (uint32)
// 4 bytes: schema ID (uint32)
// 2 bytes: attribute count (uint16)
// For each attribute:
//   2 bytes: attribute index
//   2 bytes: attribute length
//   N bytes: attribute value
// BBS+ signature:
//   48 bytes: A (G1 point, compressed)
//   32 bytes: e (scalar)
//   32 bytes: s (scalar)
// Revocation:
//   32 bytes: handle
//   96 bytes: witness
```